

ARTÍCULO DE INVESTIGACIÓN

Análisis comparativo del rendimiento de procesamiento de datos en Apache Hadoop MapReduce, Apache Spark y Apache Flink sobre un entorno virtualizado Debian GNU/Linux

Comparative Analysis of Data Processing Performance in Apache Hadoop MapReduce, Apache Spark and Apache Flink on a Virtualized Debian GNU/Linux Environment

Luis Guillermo Huarcaya Tito

Universidad Nacional de Cañete, Lima, Perú

*Corresponsal: 2301010150@undc.edu.pe

Abstract

Background: This paper presents a comparative performance analysis of three widely used Big Data processing frameworks: Apache Hadoop MapReduce, Apache Spark, and Apache Flink, deployed on a single-node virtualized environment running Debian GNU/Linux 13 (Trixie) under VirtualBox.

Methods: A WordCount benchmark was executed on datasets of 500 MB and 2 GB. Execution time (seconds) and memory consumption (GB) were recorded using a Python orchestration script with the psutil library. Results were exported to CSV and visualized in Microsoft Power BI.

Results: Apache Flink achieved the lowest execution times (2.38 s for 500 MB; 2.26 s for 2 GB), outperforming Hadoop MapReduce (27.17 s; 97.7 s) and Apache Spark (41.04 s; 133.97 s) under the tested conditions.

Conclusions: In a standalone, single-node scenario without parallelism, Flink's native streaming engine provides significant latency advantages. Spark's initialization overhead reduces its competitiveness in batch tasks of this scale, while MapReduce's disk-based I/O model remains the limiting factor.

Keywords: Big Data, Apache Hadoop, Apache Spark, Apache Flink, benchmarking, performance analysis, distributed computing.

Resumen

Antecedentes: Este artículo presenta un análisis comparativo del rendimiento de tres frameworks de procesamiento de Big Data ampliamente utilizados: Apache Hadoop MapReduce, Apache Spark y Apache Flink, desplegados en un entorno virtualizado de un solo nodo con Debian GNU/Linux 13 (Trixie) bajo VirtualBox.

Métodos: Se ejecutó un benchmark de conteo de palabras (WordCount) sobre datasets de 500 MB y 2 GB. El tiempo de ejecución (segundos) y el consumo de memoria (GB) fueron registrados mediante un script orquestador en Python con la librería psutil. Los resultados fueron exportados a CSV y visualizados en Microsoft Power BI.

Resultados: Apache Flink obtuvo los menores tiempos de ejecución (2,38 s para 500 MB; 2,26 s para 2 GB), superando a Hadoop MapReduce (27,17 s; 97,7 s) y a Apache Spark (41,04 s; 133,97 s) bajo las condiciones evaluadas.

Conclusiones: En un entorno standalone de nodo único sin paralelismo distribuido, el motor de streaming nativo de Flink ofrece ventajas de latencia significativas. El overhead de inicialización de Spark reduce su competitividad en tareas batch de esta escala, mientras que el modelo de E/S en disco de MapReduce continúa siendo su principal limitante.

Palabras clave: Big Data, Apache Hadoop, Apache Spark, Apache Flink, benchmarking, análisis de rendimiento, computación distribuida.

1. INTRODUCCIÓN

El volumen de datos generados globalmente crece de manera exponencial, impulsado por el auge de dispositivos conectados, redes sociales y sistemas de información empresarial [1]. Ante esta realidad, el procesamiento distribuido de grandes volúmenes de datos conocido como Big Data se ha convertido en una disciplina fundamental de la ingeniería de sistemas e informática [2].

Apache Hadoop MapReduce fue durante años el estándar de facto para el procesamiento masivo de datos, gracias a su capacidad de distribuir tareas a través de clústeres de nodos commodity [3]. Sin embargo, su modelo de lectura y escritura en disco entre fases Map y Reduce genera una latencia considerable, lo que motivó el desarrollo de alternativas en memoria como Apache Spark [4] y Apache Flink [5].

Apache Spark, propuesto por Zaharia et al. en 2012, introdujo el concepto de Resilient Distributed Datasets (RDD) para mantener datos en memoria de forma tolerante a fallos, logrando mejoras de hasta 100 veces frente a MapReduce en operaciones iterativas [4]. Por su parte, Apache Flink unificó el procesamiento batch y streaming en un solo motor basado en flujos de datos continuos (dataflows), con manejo explícito de la dimensión temporal [5].

La elección del framework adecuado para una tarea de procesamiento de datos depende de múltiples factores: el tipo de carga de trabajo, los recursos de hardware disponibles, la latencia tolerable y el ecosistema de herramientas. En este contexto, la comparación experimental en entornos controlados resulta indispensable para orientar decisiones técnicas fundamentadas.

El presente estudio evalúa el rendimiento de los tres frameworks mencionados en un escenario de nodo único virtualizado sobre Debian GNU/Linux, con el objetivo de identificar diferencias de rendimiento reproducibles bajo condiciones homogéneas de hardware y software.

2. MÉTODO

2.1 Entorno experimental

Las pruebas se realizaron sobre una máquina virtual con las siguientes especificaciones:

Parámetro	Especificación
Hipervisor	Oracle VirtualBox 7.x
Sistema operativo	Debian GNU/Linux 13 Trixie (64-bit)
RAM asignada	10 GB
Almacenamiento virtual	35 GB (VDI dinámico)
Java	OpenJDK 21
Apache Hadoop	3.3.6 (modo pseudo-distribuido)
Apache Spark	3.5.1 (modo local[*])
Apache Flink	1.20.0 (modo standalone)
Python	3.x + psutil

Tabla 1. Especificaciones del entorno experimental.

2.2 Datasets de prueba

Se generaron dos datasets sintéticos de texto aleatorio, compuestos por palabras seleccionadas aleatoriamente de un vocabulario de 14 términos del dominio Big Data. Los tamaños seleccionados permiten observar el comportamiento de cada framework en cargas de trabajo medianas y grandes:

Dataset	Tamaño	Iteraciones de generación
Dataset A	~500 MB	4 000 000 líneas × 20 palabras
Dataset B	~2 GB	16 000 000 líneas × 20 palabras

Tabla 2. Características de los datasets utilizados.

2.3 Tarea benchmark: WordCount

La tarea WordCount fue seleccionada como benchmark estándar por ser representativa de operaciones de conteo y agrupación, y por contar con implementaciones nativas en los tres frameworks evaluados. La tarea consiste en contar la frecuencia de aparición de cada palabra en el dataset de entrada y emitir los pares (palabra, conteo) como salida.

Cada framework ejecutó la misma tarea lógica, garantizando comparabilidad: MapReduce utilizó el jar de ejemplos incluido en Hadoop 3.3.6, Spark ejecutó un script PySpark con transformaciones flatMap/reduceByKey sobre el sistema de archivos local, y Flink utilizó el jar de WordCount incluido en su distribución de ejemplos.

2.4 Métricas e instrumentación

Las métricas registradas para cada ejecución fueron el tiempo de ejecución total en segundos, medido con la función `time.time()` de Python entre el inicio y la finalización del subprocesso correspondiente, y el consumo de memoria RAM aproximado, medido como la variación de memoria usada por el sistema antes y después de cada job, mediante la librería `psutil`.

Cada combinación framework-dataset fue ejecutada una vez dada la restricción de recursos de una VM de nodo único. Los resultados fueron almacenados en un archivo CSV estructurado con las columnas Framework, Dataset, Tiempo_s y RAM_GB, y posteriormente visualizados en Microsoft Power BI.

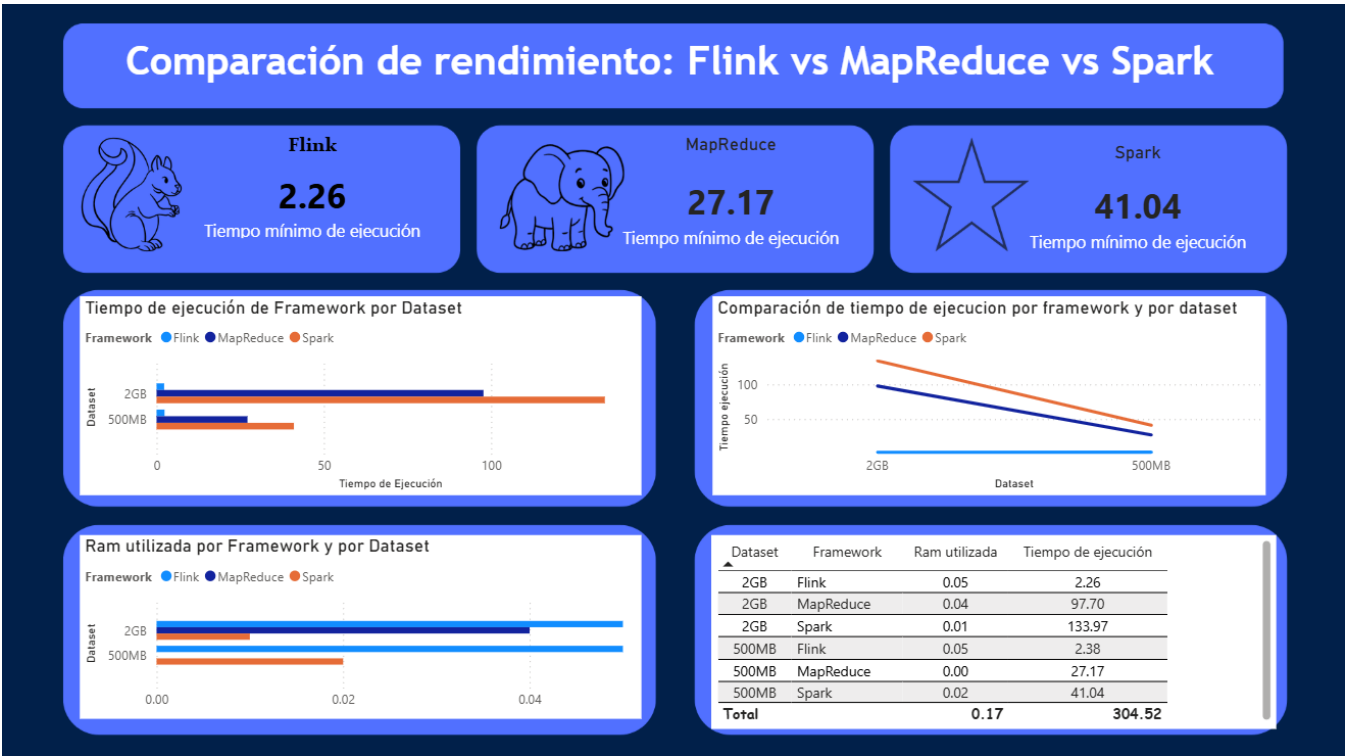


Fig. 1. Dashboard de resultados en Microsoft Power BI: tiempo de ejecución por framework y tamaño de dataset.

3. RESULTADOS

3.1 Tiempos de ejecución

La Tabla 3 presenta los tiempos de ejecución registrados para cada framework en los dos tamaños de dataset evaluados. Apache Flink obtuvo consistentemente los menores tiempos en ambos escenarios, seguido de MapReduce y finalmente Spark.

Framework	Dataset 500 MB (s)	Dataset 2 GB (s)	Variación (×)
Apache Hadoop MapReduce	27,17	97,70	3,60×
Apache Spark	41,04	133,97	3,26×
Apache Flink	2,38	2,26	0,95×

Tabla 3. Tiempos de ejecución (WordCount) por framework y tamaño de dataset.

Un hallazgo notable es que Flink mantuvo un tiempo de ejecución prácticamente constante entre el dataset de 500 MB (2,38 s) y el de 2 GB (2,26 s), lo que sugiere que en este escenario el overhead de inicialización del cluster domina sobre el tiempo real de procesamiento, independientemente del volumen de datos.

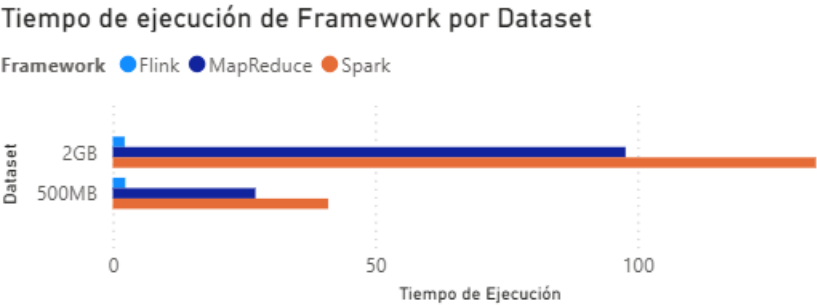


Fig. 2. Gráfico de barras agrupadas: tiempo de ejecución (s) por framework y dataset en Power BI.

3.2 Consumo de memoria RAM

Framework	Dataset 500 MB (GB)	Dataset 2 GB (GB)	Observación
Apache Hadoop MapReduce	0,00	0,04	Bajo consumo en memoria
Apache Spark	0,02	0,01	Variable por GC de JVM
Apache Flink	0,05	0,05	Estable entre datasets

Tabla 4. Consumo de memoria RAM adicional durante la ejecución del benchmark.

Las diferencias de consumo de memoria son pequeñas en términos absolutos dado el entorno de nodo único, y reflejan principalmente el overhead del proceso JVM de cada framework. Flink muestra el consumo más consistente (0,05 GB en ambos datasets), mientras que MapReduce registra un consumo cercano a cero para el dataset de 500 MB, al delegar gran parte del procesamiento al sistema de archivos.

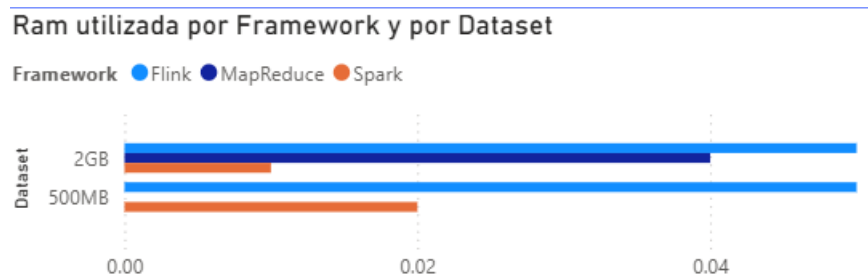


Fig. 3. Gráfico de barras: consumo de memoria RAM (GB) por framework y dataset en Power BI.

4. DISCUSIÓN

Los resultados obtenidos presentan diferencias notables respecto a estudios previos realizados en entornos distribuidos multi-nodo. López et al. [6] reportaron que Spark supera a Hadoop en un 251% en datasets de 1 GB con clústeres multi-nodo, y que Flink presentó tiempos superiores a Spark en batch processing distribuido [6]. Sin embargo, en el entorno de nodo único evaluado en este estudio, Flink obtuvo ventajas significativas, lo que indica que su motor de streaming nativo minimiza el overhead de orquestación en escenarios sin paralelismo real.

La superioridad de Flink en este contexto se explica por la ausencia de fases de shuffle explícitas y la capacidad del motor de pipelines continuos de procesar datos sin materializar resultados intermedios en disco. MapReduce, en contraste, escribe los resultados de la fase Map en disco antes de iniciar Reduce, lo que genera latencia adicional proporcional al tamaño del dataset [3].

Spark presentó los tiempos más altos en este benchmark, contrario a su reputación de framework de alto rendimiento. Esto se debe principalmente al overhead de inicialización del SparkContext y al gestor de particiones en modo local[*], que en datasets pequeños y medianos puede superar al tiempo real de procesamiento. Akil et al. [7] observaron un comportamiento similar al comparar Spark y Flink desde una perspectiva de usabilidad, indicando que el rendimiento depende fuertemente del caso de uso específico y la configuración del entorno.

La consistencia de Flink entre los dos tamaños de dataset (2,38 s vs 2,26 s) sugiere que el cuello de botella en este escenario no es el volumen de datos sino el tiempo de arranque del cluster standalone. En un entorno distribuido real con múltiples nodos, los resultados probablemente mostrarían diferencias más pronunciadas entre frameworks, favoreciendo a Spark en operaciones batch iterativas.

5. CONCLUSIONES

El presente estudio demostró experimentalmente que el rendimiento relativo de los frameworks de Big Data varía significativamente según el entorno de despliegue. En un entorno virtualizado de nodo único sobre Debian GNU/Linux, Apache Flink logró el menor tiempo de ejecución en la tarea WordCount para ambos tamaños de dataset evaluados, superando a Apache Hadoop MapReduce y Apache Spark.

MapReduce, aunque más lento que Flink, mostró un escalado lineal predecible con el tamaño del dataset (3,60×), coherente con su modelo de procesamiento en disco. Spark, diseñado para entornos distribuidos con múltiples nodos y alta disponibilidad de memoria, no demostró ventajas en este escenario de nodo único, siendo el más lento de los tres frameworks evaluados.

Para trabajos futuros se recomienda extender el análisis a entornos multi-nodo con clústeres de al menos tres nodos, incorporar tareas adicionales como TeraSort y operaciones de join, y evaluar el impacto de diferentes configuraciones de paralelismo y memoria en cada framework. Asimismo, sería valioso incluir métricas de throughput (registros procesados por segundo) y consumo energético para una evaluación más integral.

CONTRIBUCIÓN DEL AUTOR

Luis Guillermo Huarcaya Tito: conceptualización, metodología, software, instalación del entorno, ejecución de experimentos, análisis de resultados, visualización y redacción del manuscrito original.

FINANCIAMIENTO

Este trabajo fue realizado como proyecto de investigación aplicada del curso de Big Data de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional de Cañete, sin financiamiento externo.

ÉTICA

Este estudio no involucra datos de seres humanos ni animales. Todos los datos utilizados son sintéticos, generados computacionalmente para fines de benchmarking académico.

DISPONIBILIDAD DE LOS DATOS

Los scripts de instalación, benchmark y generación de datasets, así como el archivo CSV de resultados, están disponibles bajo solicitud al autor correspondiente.

AGRADECIMIENTOS

El autor agradece a la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional de Cañete y al Mg. Joel Linder Vilca Pizarro por el apoyo y orientación durante el desarrollo de este trabajo.

CONFLICTO DE INTERESES

El autor declara que no existe ningún conflicto de intereses.

REFERENCIAS

- [1] Badshah, A., Daud, A., Alharbey, R., "Big data applications: overview, challenges and future," Full Access, Rev. 57, p. 290, Ago. 2024, doi: 10.1016/J.ACALIB.2023.102741. Disponible en: <https://doi.org/10.1016/j.acalib.2023.102741>
- [2] Akil, B., Zhou, Y., Röhm, U., "On the usability of Hadoop MapReduce, Apache Spark & Apache Flink for data science," in 2017 IEEE International Conference on Big Data (Big Data), pp. 303–310, 2017, doi: 10.1109/BigData.2017.8257938. Disponible en: <https://ieeexplore.ieee.org/document/8257938/>

- [3] Dean, J., Ghemawat, S., "MapReduce: Simplified data processing on large clusters," Commun. ACM, vol. 51, no. 1, pp. 107–113, 2008, doi: 10.1145/1327452.1327492. Disponible en: https://www.researchgate.net/publication/220851866_MapReduce_Simplified_Data_Processing_on_Large_Clusters
- [4] Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., Stoica, I., "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in 9th USENIX Symp. Networked Syst. Design and Implementation (NSDI 12), pp. 15–28, 2012. Disponible en: <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>
- [5] Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., Tzoumas, K., "Apache Flink: Stream and batch processing in a single engine," IEEE Data(base) Engineering Bulletin, vol. 36, no. 4, pp. 28–38, 2015. Disponible en: <https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf>
- [6] López et al., "Performance Evaluation of Apache Hadoop, Spark, and Flink for Batch Processing of Big Data: A Comparative Analysis," in IEEE Conference on Big Data, 2024, doi: 10.1109/BigData.2024.10718602. Disponible en: <https://ieeexplore.ieee.org/iel8/10718331/10718367/10718602.pdf>
- [7] Marcu, O.-C., Costan, A., Antoniu, G., Pérez-Hernández, M. S., "Spark versus Flink: Understanding performance in big data analytics frameworks," in 2016 IEEE International Conference on Cluster Computing (CLUSTER), pp. 433–442, 2016. Disponible en: <https://arxiv.org/pdf/2209.10637>
- [8] Jha, S., Qiu, J., Luckow, A., Mantha, P., Fox, G. C., "A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench," Journal of Big Data, 2020, doi: 10.1186/s40537-020-00388-5. Disponible en: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00388-5>